MIND MATRIX

NAME: MOHAMMED SAHIL R

EMAIL ID: mohammedsahilr39@gmail.com

SESSION: 10

REPORT ON: TASK REPORT

TASK NO: 01-03

TASK 1 – DESIGN YOUR OWN DATA MODEL AND BLUEPRINT CLASS

Introduction

This task focuses on understanding object-oriented programming concepts in Kotlin such as classes, constructors, and object instances. Students learn how to design a data model that represents a real-world entity and how that model can be used inside an Android application.

Key Concepts

What is a Class in Kotlin?

A class in Kotlin is a blueprint used to create objects. It defines properties and functions that describe the behavior and characteristics of an object.

Difference Between Class and Object

A **class** is a blueprint, while an **object** is an instance of that class containing actual values.

Primary and Secondary Constructors

A **primary constructor** initializes class properties directly in the class header, while a **secondary constructor** provides additional ways to create objects.

Init Block

An **init block** is executed when an object is created. It is commonly used to perform initialization or validation.

Member Functions

Member functions are functions defined inside a class that describe the behavior of the object.

Example Data Class

```
data class Product(
    val name: String,
    val price: Double,
    val category: String
)
```

Class Design – Fitness App Workout

```
class Workout(
    val name: String,
    val duration: Int,
    val caloriesBurned: Int,
    var isCompleted: Boolean
) {

    init {
        println("Workout created: $name")
    }

    fun difficultyLevel(): String {
        return if (duration < 20) "Easy" else "Hard"
    }

    fun toggleCompletion() {
        isCompleted = !isCompleted
    }
}
```

UML Style Diagram

```
-------------------------
|       Workout         |
-------------------------
| name: String          |
| duration: Int         |
| caloriesBurned: Int   |
| isCompleted: Boolean  |
-------------------------
| difficultyLevel()     |
| toggleCompletion()    |
-------------------------
```

Object Instances

```
val workout1 = Workout("Morning Yoga", 15, 100, false)
val workout2 = Workout("HIIT Training", 30, 300, false)

println(workout1.difficultyLevel())
workout2.toggleCompletion()
```

Compose Integration Plan

This data model can be displayed in a Compose UI using a **LazyColumn** that lists workouts. Each item can show the workout name, duration, and calories burned. A button can be added to mark the workout as completed, which updates the UI using state management.

Reflection

Designing a class helps developers organize data and behavior clearly before implementing UI logic. Modeling real-world entities improves code readability and maintainability. It also allows developers to reuse objects easily across different parts of an application.

TASK 2 – FUNCTIONAL PROGRAMMING WITH LAMBDAS AND HIGHER ORDER FUNCTIONS

Introduction

This task introduces functional programming concepts in Kotlin such as lambdas and higher-order functions. These concepts allow developers to write more reusable and concise code.

Key Concepts

Higher Order Function

A higher-order function is a function that takes another function as a parameter or returns a function.

Lambda Expression

A lambda is a small anonymous function that can be passed as an argument.

Function Type

A function type describes the parameters and return type of a function.

Example:

(Int, Int) -> Int

Inline Function

Inline functions reduce overhead by inserting the function code directly during compilation.

Lambda Example

```
val numbers = listOf(1,2,3,4,5)
val evenNumbers = numbers.filter { it % 2 == 0 }
println(evenNumbers)
```

Higher Order Function Example

```
fun applyOperation(a: Int, b: Int, operation: (Int, Int) -> Int): Int {
    return operation(a, b)
}

val result = applyOperation(5, 3) { x, y -> x + y }
println(result)
```

Mini Use Case – Task Filtering

Data Model

```
data class Task(
    val title: String,
    val isUrgent: Boolean
)
```

Pseudo Code

```
tasks.filter { task -> task.isUrgent }
```

Compose Lambda Usage

Common lambda usage in Compose:

- Button(onClick = { })
- Modifier.clickable { }
- LazyColumn item click handling

Example:

```
Button(onClick = { total++ }) {
   Text("Add")
}
```

Function Type Alias Example

```
typealias DiscountRule = (Double) -> Double
```

Example usage:

```
fun applyDiscount(price: Double, rule: DiscountRule): Double {
   return rule(price)
}
```

Reflection

Using lambdas improves code reusability and readability. They allow developers to pass behavior as parameters, making the code more flexible. Lambdas are also heavily used in Compose for handling events such as button clicks and UI interactions.

TASK 3 – MINI PROJECT: COMPOSE APP FEATURE COMBINING OOP AND LAMBDAS

App Theme

Feature: **Book Library Application**

This feature manages a list of books and allows users to mark books as favorites or filter them by genre. The application demonstrates how object-oriented programming and lambda expressions work together.

Class Design

```
data class Book(
    val title: String,
    val author: String,
    val genre: String,
    var isFavorite: Boolean
) {

    fun toggleFavorite() {
        isFavorite = !isFavorite
    }

    fun displayInfo(): String {
        return "$title by $author"
    }

    companion object {
        const val MAX_BOOKS = 100
    }
}
```

Object List

```
val books = listOf(
    Book("Atomic Habits","James Clear","Self Help",false),
```

```
    Book("Clean Code","Robert Martin","Programming",true),
    Book("Kotlin in Action","Dmitry Jemerov","Programming",false)
)
```

Lambda Filter Example

```
val programmingBooks = books.filter { it.genre == "Programming" }
```

UI Sketch

```
-------------------------
| Book Library          |
-------------------------
| Atomic Habits         |
| Clean Code            |
| Kotlin in Action      |
-------------------------
[ Favorite Button ]
```

Architecture Flow Diagram

```
Book Data Model
      |
      v
Lambda Filter Logic
      |
      v
Compose UI List Display
```

Edge Cases

Case	Handling
Empty book list	Show message "No books available"
Null object	Safe call operator
Invalid user input	Display error message

Reflection

This task combines object-oriented programming and functional programming concepts. Using classes helps structure data models, while lambdas simplify filtering and event handling. Combining both approaches improves application flexibility and code organization.